



AIUB Sport Portal Management System

Course Name

ADVANCE DATABASE MANAGEMENT SYSTEM

Section: C

Group Members

Name	ID	Contribution
MD. Nashib Khan (Project Lead)	22-49873-3	Introduction, Project proposal ,Scenario description, User Interface design, PL/SQL, Database design (25%)
MD Momen Sha	24-56434-1	Idea Generation, User Interface, Advance PL/SQL, Database design, Schema Diagram , Coding(frontend &backend) (25%)
MD. Al Amin	22-48081-2	ER Diagram, Normalization, Schema Diagram, Basic PL/SQL, Advance PL/SQL (25%)
MD. Sharif Iqbal	22-49924-3	Project proposal, Table Creation using SQL, Basic PL/SQL , Coding(frontend), Schema Diagram (25%)
Pranto Sen	23-52029-2	Introduction, Coding(Backend) ,Data Insertion Using SQL, Schema Diagram ,Conclusion . (25%)

Contents

Introduction	3
Project Proposal	3
User Interface Planning	4
Scenario Description	8
ER Diagram	9
Normalization	10
Schema Diagram	15
Table creation using SQL	16
Data Insertion Using SQL	21
Basic PL/SQL	26
Advance PL/SQL	36
Conclusion	50

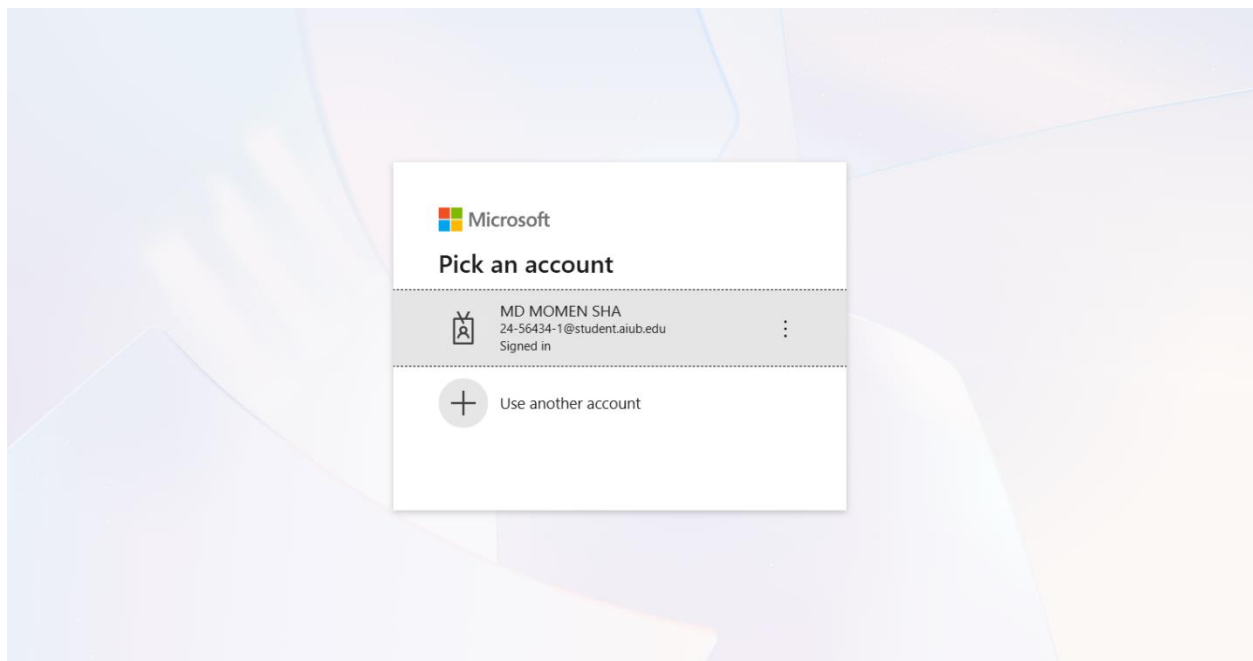
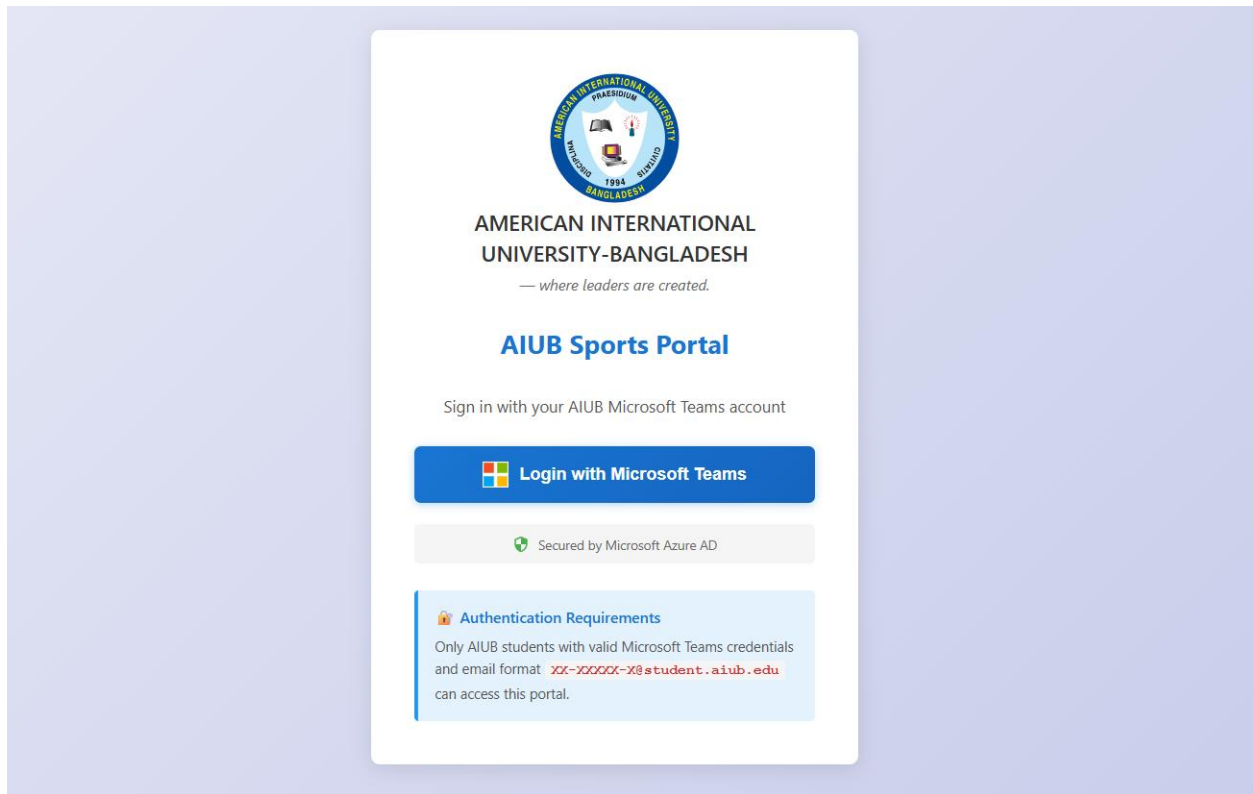
Introduction:

The AIUB Sports Portal is a web-based platform designed to make sports activities at American International University–Bangladesh easier, faster, and more organized. Every semester, many students participate in tournaments, but registration is usually manual, time-consuming, and difficult to manage. This project solves that problem by creating a digital system where students can log in with their official AIUB accounts, complete their profiles, view ongoing tournaments, and register for games. Admins can also create tournaments, manage participants, and handle all sports-related tasks from one place. In simple terms, the portal brings the entire sports management process online so both students and AIUB authorities can save time and reduce mistakes.

Project Proposal:

This project proposes the development of a complete sports management web portal for AIUB that integrates authentication, tournament management, and game registration within a secure and intuitive interface. The goal is to eliminate the traditional paperwork-based registration system and replace it with a reliable digital system that ensures accuracy, safety, and real-time data handling. The portal will allow students to log in using Microsoft Azure Active Directory, which guarantees identity verification and restricts access only to authentic AIUB students. Once logged in, students can complete their profiles, view ongoing tournaments, and register for specific sports categories. Administrators will be provided with a dedicated dashboard where they can create tournaments, upload tournament images, set registration deadlines, add game categories, and manage participants. To maintain fairness and data accuracy, the system includes several important business rules, such as limiting the number of name edits, locking sensitive profile fields after initial setup, and enforcing registration deadlines. The proposal supports scalable architecture built with Node.js, Express.js, Oracle Database (10g), and pure HTML/CSS/JavaScript on the frontend. The platform ensures secure communication, strict input validation, and structured database design. By integrating PL/SQL stored procedures, triggers, and sequences, the system achieves high data integrity and reliable automation. Overall, the AIUB Sports Portal aims to provide AIUB students and admins with professional, automated, transparent, and trustworthy sports management experience.

User Interface Planning:



 **AIUB**
SPORTS PORTAL

Registration

Welcome MD Momen Sha

Dashboard

Registration

Featured

Messages

Bug Report

Dear Students & Faculty members, Welcome to the new AIUB Sports Portal. This platform is designed to make your sports activities easier to access, manage, and enjoy. Stay updated, stay connected, and be a part of AIUB's growing sports community.

Edit Profile

Player Name

MD Momen Sha

Program

Undergraduate

Player ID

24-56434-1

Department

CSE

Player Email

24-56434-1@student.aiub.edu

Number

01616807177

Gender

Male

Blood Group

A+

No tournaments available for registration at the moment.

Registration Closed

 **AIUB**
SPORTS PORTAL

Registration

Welcome Admin

Existing Tournaments

Create Tournament

Messages

Bug Report

Admin Dashboard - Manage tournaments and oversee sports activities

Existing Tournaments

No tournaments created yet.


AIUB
SPORTS PORTAL

Registration

Welcome Admin

Existing Tournaments

+ Create Tournament

Messages

Bug Report

Admin Dashboard - Manage tournaments and oversee sports activities

Create New Tournament

Tournament Title

Registration Deadline

Tournament Photo (Optional)

Choose File No file chosen

Tournament Description (Optional)

Games

No games added yet.

+ Add game

Cancel

Create Tournament

Admin Dashboard - Manage tournaments and oversee sports activities

Create New Tournament

Tournament Title

Tournament Photo (Optional)

Choose File No file chosen

Tournament Description (Optional)

Games

No games added yet.

+ Add game

Cancel

Create Tournament

Add game to this tournament

Game

Select game

Select from default list or add a custom game.

Available for

Men Women

By default, Men and Women are both allowed for all standard games.

Tournament format

Solo Duo Duo (Mixed) Squad / Team (NVN)

Logic: Chess = only Solo, Pool = Solo+Duo+Mixed, Ludo = Duo+Mixed, Football = Squad only, etc.

Entry fee per person (t)

100

Default is 100 taka per person. Change if needed.

Cancel Save game

Games

Badminton (Male)

Type: Solo • Fee: 100 BDT

Delete

Badminton (Female)

Type: Solo • Fee: 100 BDT

Delete

Badminton (Male)

Type: Duo • Fee: 100 BDT

Delete

Badminton (Female)

Type: Duo • Fee: 100 BDT

Delete

Badminton (Mix)

Type: Duo (Mixed) • Fee: 100 BDT

Delete

+ Add game

Cancel

Create Tournament

Games

Showing 5 existing games. Add more or update below.

Male Category Games

Badminton (Duo)

100 BDT x

Badminton (Solo)

100 BDT x

Female Category Games

Badminton (Duo)

100 BDT x

Badminton (Solo)

100 BDT x

Mix Category Games

Badminton (Custom)

100 BDT x

+ Add game

Cancel

Update Tournament



-- Back to Dashboard

No Active Tournaments

Check back later for upcoming events!

Scenario Description

In an AIUB Sports Management System, students can register for different games offered within tournaments. A game must have at least one student registered, although students are not required to join any game. Each game belongs to a tournament, and it cannot exist without one. Tournaments are created and managed by admins. Students and admins are stored separately in the system. Students log in using Microsoft Teams and maintain a profile containing details like name, gender, department, program level, phone number, and blood group. Some information, such as gender and department, becomes locked after the first submission, while the name can only be edited a limited number of times. Each tournament includes a title, description, registration deadline, and an image. An admin can create multiple tournaments, and every tournament can have multiple games such as football, cricket, or badminton, where each game is classified under male, female, or mixed categories. Students can register for any available game as long as the tournament registration deadline has not passed. They may join multiple games but cannot register more than once for the same game. Each registration records the registration date and payment status, which may be pending, paid, or failed. Unique IDs identify all students, admins, tournaments, games, and registrations. Foreign keys ensure proper relationships among these entities. To keep data consistent, triggers and sequences automatically generate IDs and timestamps. Overall, the system represents a real-world sports portal where students take part in tournaments and games, while admins handle tournament creation and game management.

ER Diagram



Normalization

RELATION-1 : USER REGISTRATION: (User ↔ Game)

UNF:

Student_registration(student_id_, email, full_name, gender, program_level, department, blood_group, registration_id, registration_date, payment_status)

1NF:

There is no multivalued attribute. All attributes are atomic. Relation already in 1NF.

2NF:

1. student(student_id_, email, full_name, gender, program_level, department, blood_group)
2. GAME_REGISTRATIONS(registration_id, registration_date, payment_status)

Partial Dependency:

student_id, email, full_name, gender, program_level, department, blood_group
(depend only on student_id), registration_date, payment_status (depend on registration_id)

3NF:

No transitive dependency Relation already in 3NF.

Table Creation:

1. Student(**student_id_**, email, full_name, gender, program_level, department, blood_group)
2. GAME_REGISTRATIONS(**registration_id**, registration_date, payment_status)

RELATION-2 : TOURNAMENT_CREATION

UNF:

tournament_creation(a_id, admin_name, admin_email,created_at tournament_id, title, photo_url, registration_deadline, status, description, created_at,CreatedBy)

1NF:

There is no multivalued attribute. Relation already in 1NF

2NF:

1. ADMINS (a_id, admin_name, admin_email,created_at)
2. TOURNAMENTS(t_id, title, photo_url, registration_deadline, status, description, created_at,CreatedBy)

Partial Dependency:

admin_name, admin_email (depend on a_id) , registration_deadline, status (depend on t_id)

3NF:

No transitive dependency Relation already in 3NF.

Table Creation:

1. ADMINS(**a_id**, admin_name, admin_email,created_at)
2. TOURNAMENTS(**t_id**, title, photo_url,registration_deadline, status,description, created_at,CreatedBy)

RELATION-3 : TOURNAMENT_have_Tournament_Games:

UNF:

TOURNAMENTS_HAVE_GAMES(t_id, title, photo_url, registration_deadline, status, created_by, description, g_id, category, game_name, game_type, fee_per_person, created_at)

1NF:

There is no multivalued attribute. All attributes are atomic. Relation already in 1NF

2NF:

1. TOURNAMENTS(t_id (PK), title, photo_url, registration_deadline, status, created_by, description)
2. TOURNAMENT_GAMES(g_id (PK), t_id (FK), category, game_name, game_type, fee_per_person)

Composite key: (t_id, g_id)

Partial dependencies:

title, photo_url, registration_deadline, status, created_by, description depends on t_id

category, game_name, game_type, fee_per_person depends on g_id

3NF:

No transitive dependency Relation already in 3NF.

Table Creation:

1. TOURNAMENTS (t_id, title, photo_url, registration_deadline, status, description, created_at, CreatedBy)
2. TOURNAMENT_GAMES(g_id, category, game_name, game_type, fee_per_person, created_at)

RELATION-4 : Game_Registrations ↔ Tournament_Games

UNF:

GAME_REGISTRATION (registration_id, registration_deadline, PaymentStatus g_id , category, game_name, game_type, fee_per_person, created_at)

1NF:

There is no multivalued attribute. All attributes are atomic. Relation already in 1NF.

2NF:

1. Game_Registrations (registration_id, PaymentStatus, RegistrationDate)
2. TOURNAMENT_GAMES(g_id , category,game_name,game_type, fee_per_person)

Composite PK = (reg_id, g_id)

Partial dependencies: registration_deadline, PaymentStatus (reg_id)

category, game_name, game_type, fee_per_person, created_at (g_id)

3NF:

No transitive dependency Relation already in 3NF.

Table Creation:

1. Game_Registrations (**registration_id**, PaymentStatus, RegistrationDate)
2. TOURNAMENT_GAMES(**g_id** , category, game_name, game_type, fee_per_person)

Temporary Tables:

1. **student_id**, email, full_name, gender, program_level, department, blood_group
- ~~2. registration_id, registration_date, payment_status, student_id (FK)~~
3. **a_id**, admin_name, admin_email, created_at
- ~~4. t_id, title, photo_url, registration_deadline, status, description, created_at, CreatedBy, a_id(FK)~~
5. **t_id**, title, photo_url, registration_deadline, status, description, created_at, CreatedBy, a_id(FK)
- ~~6. g_id, category, game_name, game_type, fee_per_person, created_at, t_id(FK)~~
7. **registration_id**, registration_date, payment_status, student_id (FK), t_id(FK)
8. **g_id**, category, game_name, game_type, fee_per_person, created_at, t_id(FK)

Final Tables:

1. **student_id**, email, full_name, gender, program_level, department, blood_group
2. **a_id**, admin_name, admin_email, created_at
3. **t_id**, title, photo_url, registration_deadline, status, description, created_at, CreatedBy, a_id(FK)
4. **registration_id**, registration_date, payment_status, student_id (FK), t_id(FK)
5. **g_id**, category, game_name, game_type, fee_per_person, created_at, t_id(FK)

Schema Diagram

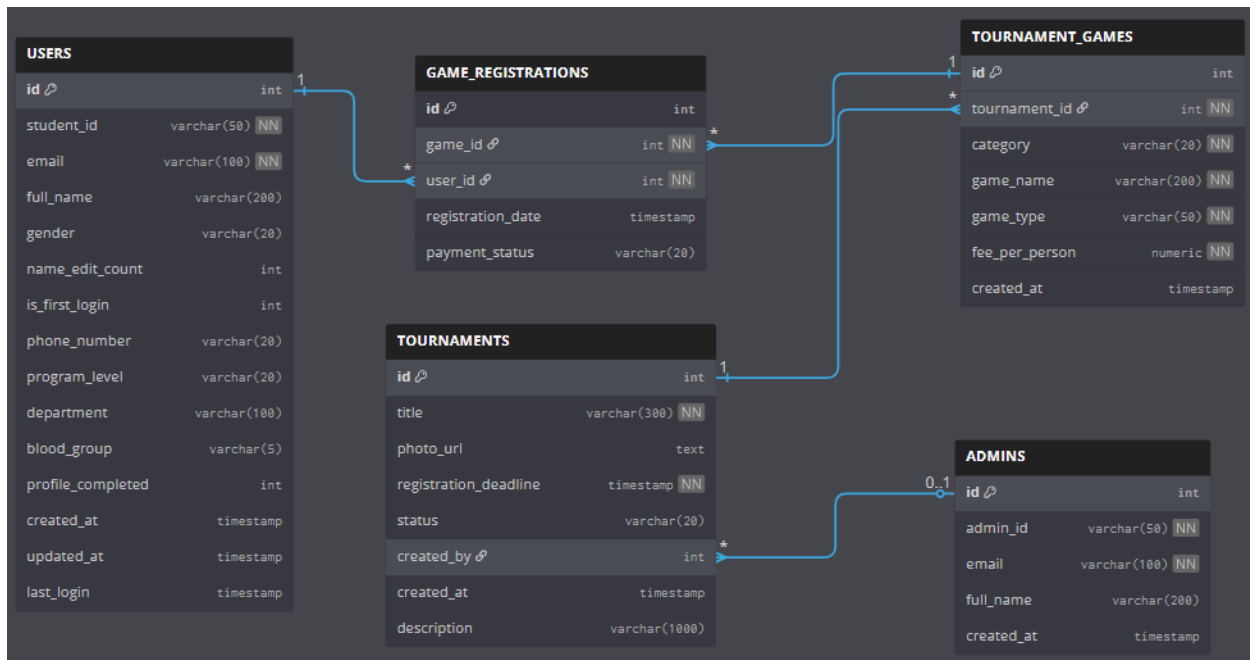
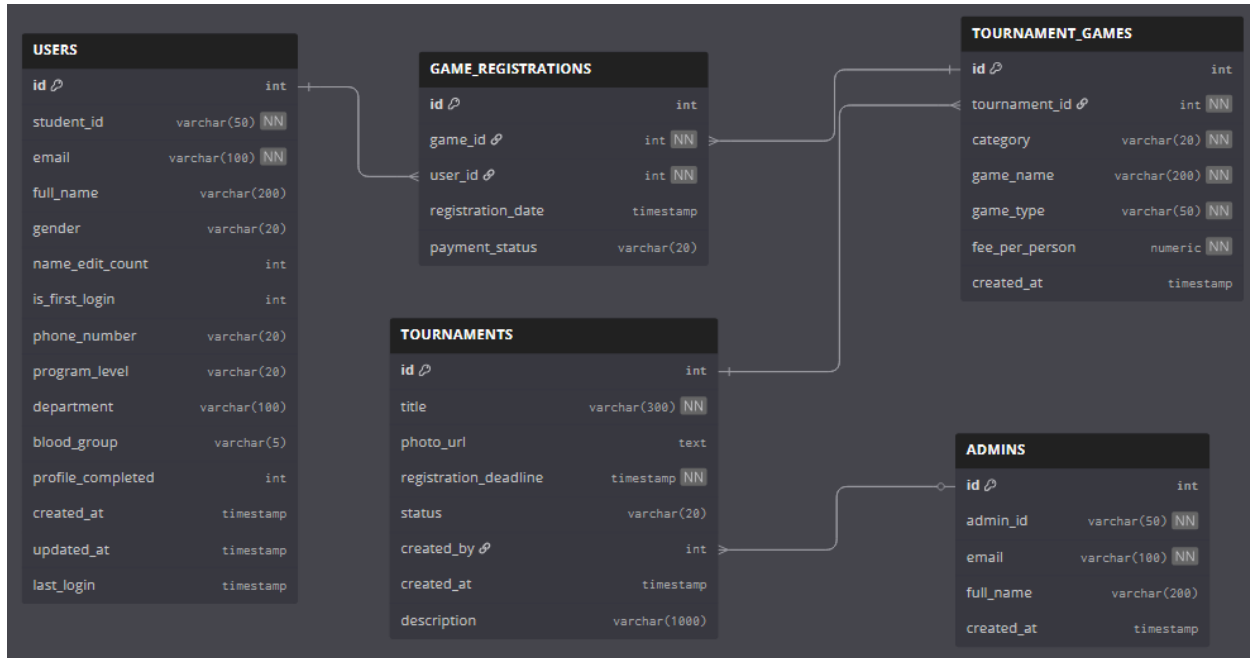


Table creation using SQL

Users Table

```
CREATE TABLE users (  
    id NUMBER PRIMARY KEY,  
    student_id VARCHAR2(50) NOT NULL UNIQUE,  
    email VARCHAR2(100) NOT NULL UNIQUE,  
    full_name VARCHAR2(200),  
    gender VARCHAR2(20),  
    name_edit_count NUMBER DEFAULT 0,  
    is_first_login NUMBER(1) DEFAULT 1,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    last_login TIMESTAMP,  
    phone_number VARCHAR2(20),  
    program_level VARCHAR2(20),  
    department VARCHAR2(100),  
    blood_group VARCHAR2(5),  
    profile_completed NUMBER(1) DEFAULT 0,  
    CONSTRAINT chk_gender CHECK (gender IN ('Male', 'Female', 'Other', NULL)),  
    CONSTRAINT chk_edit_count CHECK (name_edit_count >= 0 AND name_edit_count <= 3),  
    CONSTRAINT chk_first_login CHECK (is_first_login IN (0, 1)),  
    CONSTRAINT chk_program_level CHECK (program_level IN ('Undergraduate', 'Postgraduate', NULL)),  
    CONSTRAINT chk_blood_group CHECK (blood_group IN ('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-', NULL))  
);
```


ORACLE Database Express Edition

User: WEBUSER

Home > Object Browser

Tables

ADMINIS
GAME_REGISTRATIONS
TOURNAMENTS
TOURNAMENT_GAMES
USERS

USERS

Create

Table Data Indexes Model Constraints Grants Statistics UI Defaults Triggers Dependencies SQL

Add Column Modify Column Rename Column Drop Column Rename Copy Drop Truncate Create Lookup Table

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER	No	-	1
STUDENT_ID	VARCHAR2(50)	No	-	-
EMAIL	VARCHAR2(100)	No	-	-
FULL_NAME	VARCHAR2(200)	Yes	-	-
GENDER	VARCHAR2(20)	Yes	-	-
NAME_EDIT_COUNT	NUMBER	Yes	0	-
IS_FIRST_LOGIN	NUMBER(1,0)	Yes	1	-
CREATED_AT	TIMESTAMP(6)	Yes	CURRENT_TIMESTAMP	-
UPDATED_AT	TIMESTAMP(6)	Yes	CURRENT_TIMESTAMP	-
LAST_LOGIN	TIMESTAMP(6)	Yes	-	-
PHONE_NUMBER	VARCHAR2(20)	Yes	-	-
PROGRAM_LEVEL	VARCHAR2(20)	Yes	-	-
DEPARTMENT	VARCHAR2(100)	Yes	-	-
BLOOD_GROUP	VARCHAR2(6)	Yes	-	-
PROFILE_COMPLETED	NUMBER(1,0)	Yes	0	-

1 - 15

Language: en-us

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

PC
lear

Search

12:42 AM
12/15/2025

Admins Table

CREATE TABLE admins (

id NUMBER PRIMARY KEY,

admin_id VARCHAR2(50) NOT NULL UNIQUE,

email VARCHAR2(100) NOT NULL UNIQUE,

full_name VARCHAR2(200),

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);

ORACLE Database Express Edition

User: WEBUSER

Home > Object Browser

Tables

ADMINIS
GAME_REGISTRATIONS
TOURNAMENTS
TOURNAMENT_GAMES
USERS

ADMINIS

Create

Table Data Indexes Model Constraints Grants Statistics UI Defaults Triggers Dependencies SQL

Add Column Modify Column Rename Column Drop Column Rename Copy Drop Truncate Create Lookup Table

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER	No	-	1
ADMIN_ID	VARCHAR2(50)	No	-	-
EMAIL	VARCHAR2(100)	No	-	-
FULL_NAME	VARCHAR2(200)	Yes	-	-
CREATED_AT	TIMESTAMP(6)	Yes	CURRENT_TIMESTAMP	-

1 - 5

Language: en-us

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

0°C
lear

Search

12:42 AM
12/15/2025

Tournaments Table

```
CREATE TABLE tournaments (  
    id NUMBER PRIMARY KEY,  
    title VARCHAR2(300) NOT NULL,  
    photo_url CLOB,  
    registration_deadline TIMESTAMP NOT NULL,  
    status VARCHAR2(20) DEFAULT 'ACTIVE',  
    created_by NUMBER REFERENCES admins(id),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT chk_tournament_status CHECK (status IN ('ACTIVE', 'CLOSED',  
'COMPLETED'))  
);
```

The screenshot displays the Oracle Database Express Edition interface. The left sidebar shows a tree view with the following objects: ADMINIS, GAME_REGISTRATIONS, TOURNAMENTS (highlighted), TOURNAMENT_GAMES, and USERS. The main area is titled 'TOURNAMENTS' and contains a table with the following columns and data types:

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER	No	-	1
TITLE	VARCHAR2(300)	No	-	-
PHOTO_URL	CLOB	Yes	-	-
REGISTRATION_DEADLINE	TIMESTAMP(6)	No	-	-
STATUS	VARCHAR2(20)	Yes	'ACTIVE'	-
CREATED_BY	NUMBER	Yes	-	-
CREATED_AT	TIMESTAMP(6)	Yes	CURRENT_TIMESTAMP	-
DESCRIPTION	VARCHAR2(1000)	Yes	-	-

The interface also includes a top navigation bar with 'Home', 'Logout', and 'Help' links, and a bottom status bar showing 'Language: en-us' and 'Application Express 2.1.0.00.39'.

Tournament Games Table

```
CREATE TABLE tournament_games (  
    id NUMBER PRIMARY KEY,  
    tournament_id NUMBER NOT NULL REFERENCES tournaments(id) ON DELETE  
    CASCADE,  
    category VARCHAR2(20) NOT NULL,  
    game_name VARCHAR2(200) NOT NULL,  
    game_type VARCHAR2(50) NOT NULL,  
    fee_per_person NUMBER NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT chk_game_category CHECK (category IN ('Male', 'Female', 'Mix')),  
    CONSTRAINT chk_game_type CHECK (game_type IN ('Solo', 'Duo', 'Custom'))  
);
```

The screenshot displays the Oracle Database Express Edition interface. The left sidebar shows a tree view with 'TOURNAMENT_GAMES' selected. The main area shows the table's structure with the following columns:

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER	No	-	1
TOURNAMENT_ID	NUMBER	No	-	-
CATEGORY	VARCHAR2(20)	No	-	-
GAME_NAME	VARCHAR2(200)	No	-	-
GAME_TYPE	VARCHAR2(50)	No	-	-
FEE_PER_PERSON	NUMBER	No	-	-
CREATED_AT	TIMESTAMP(6)	Yes	CURRENT_TIMESTAMP	-

The interface also includes a top navigation bar with 'HOME', 'LOGOUT', and 'HELP' links, and a bottom status bar showing the application version and copyright information.

Game Registrations Table

```
CREATE TABLE game_registrations (  
    id NUMBER PRIMARY KEY,  
    game_id NUMBER NOT NULL REFERENCES tournament_games(id) ON DELETE  
CASCADE,  
    user_id NUMBER NOT NULL REFERENCES users(id),  
    registration_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    payment_status VARCHAR2(20) DEFAULT 'PENDING',  
    CONSTRAINT chk_payment_status CHECK (payment_status IN ('PENDING', 'PAID',  
'FAILED'))  
);
```

The screenshot displays the Oracle Database Express Edition interface. On the left, a sidebar shows a tree view of database objects, including 'ADMIN', 'GAME_REGISTRATIONS', 'TOURNAMENTS', 'TOURNAMENT_GAMES', and 'USERS'. The main area is titled 'GAME_REGISTRATIONS' and contains a table structure view. The table has five columns: 'ID' (NUMBER, Primary Key), 'GAME_ID' (NUMBER), 'USER_ID' (NUMBER), 'REGISTRATION_DATE' (TIMESTAMP(6)), and 'PAYMENT_STATUS' (VARCHAR2(20)). The 'ID' column is marked as the primary key with a '1' in the 'Primary Key' column. The 'REGISTRATION_DATE' column has a default value of 'CURRENT_TIMESTAMP'. The 'PAYMENT_STATUS' column has a default value of 'PENDING'. The table is located in the '1 - 5' schema.

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER	No	-	1
GAME_ID	NUMBER	No	-	-
USER_ID	NUMBER	No	-	-
REGISTRATION_DATE	TIMESTAMP(6)	Yes	CURRENT_TIMESTAMP	-
PAYMENT_STATUS	VARCHAR2(20)	Yes	'PENDING'	-

Data Insertion Using SQL

Insert Data into Users table

```
INSERT INTO users (id, student_id, email, full_name, gender, phone_number, program_level, department, blood_group, profile_completed, name_edit_count, is_first_login)
```

```
VALUES (user_id_seq.NEXTVAL, '24-56434-1', '24-56434-1@student.aiub.edu', 'John Smith', 'Male', '01712345678', 'Undergraduate', 'CSE', 'A+', 1, 0, 0);
```

```
INSERT INTO users (id, student_id, email, full_name, gender, phone_number, program_level, department, blood_group, profile_completed, name_edit_count, is_first_login)
```

```
VALUES (user_id_seq.NEXTVAL, '23-45123-2', '23-45123-2@student.aiub.edu', 'Sarah Johnson', 'Female', '01823456789', 'Undergraduate', 'EEE', 'B+', 1, 1, 0);
```

```
INSERT INTO users (id, student_id, email, full_name, gender, phone_number, program_level, department, blood_group, profile_completed, name_edit_count, is_first_login)
```

```
VALUES (user_id_seq.NEXTVAL, '22-34567-3', '22-34567-3@student.aiub.edu', 'Robert Davis', 'Male', '01634567890', 'Postgraduate', 'BBA', 'O+', 1, 2, 0);
```

```
INSERT INTO users (id, student_id, email, full_name, gender, phone_number, program_level, department, blood_group, profile_completed, name_edit_count, is_first_login)
```

```
VALUES (user_id_seq.NEXTVAL, '21-23456-4', '21-23456-4@student.aiub.edu', 'Emily Brown', 'Female', '01745678901', 'Undergraduate', 'English', 'AB-', 1, 0, 0);
```

```
INSERT INTO users (id, student_id, email, full_name, gender, phone_number, program_level, department, blood_group, profile_completed, name_edit_count, is_first_login)
```

```
VALUES (user_id_seq.NEXTVAL, '20-12345-5', '20-12345-5@student.aiub.edu', 'Michael Wilson', 'Male', '01556789012', 'Postgraduate', 'Law', 'B-', 1, 3, 0);
```

Oracle Database Express Edition SQL Command window showing the insertion of five users into the 'users' table. The SQL commands are executed, and the results are displayed in a table.

ID	STUDENT_ID	EMAIL	FULL_NAME	GENDER	NAME_EDIT_COUNT	IS_FIRST_LOGIN	CREATED_AT	UPDATED_AT	LAST_LOGIN	PHONE_NUMBER	PROGRAM_LEVEL	DEPARTMENT	BLOOD_GROUP	PROFILE_COMPLETED
1	24-56434-1	24-56434-1@student.aiub.edu	John Smith	Male	0	0	14-DEC-25 08:11:22.828000 PM	14-DEC-25 08:11:22.828000 PM	-	01712345678	Undergraduate	CSE	A+	1
2	23-45123-2	23-45123-2@student.aiub.edu	Sarah Johnson	Female	1	0	14-DEC-25 08:12:13.074000 PM	14-DEC-25 08:12:13.074000 PM	-	01823456789	Undergraduate	EEE	B+	1
3	22-34567-3	22-34567-3@student.aiub.edu	Robert Davis	Male	2	0	14-DEC-25 08:12:36.383000 PM	14-DEC-25 08:12:36.383000 PM	-	01634567890	Postgraduate	BBA	O+	1
4	21-23456-4	21-23456-4@student.aiub.edu	Emily Brown	Female	0	0	14-DEC-25 08:12:55.065000 PM	14-DEC-25 08:12:55.065000 PM	-	01745678901	Undergraduate	English	AB-	1
5	20-12345-5	20-12345-5@student.aiub.edu	Michael Wilson	Male	3	0	14-DEC-25 08:13:13.103000 PM	14-DEC-25 08:13:13.103000 PM	-	01556789012	Postgraduate	Law	B-	1

5 rows returned in 0.02 seconds

Insert Data into Admin table

```
INSERT INTO admins (id, admin_id, email, full_name)
```

```
VALUES (admin_id_seq.NEXTVAL, 'admin001', 'admin1@aiub.edu', 'Admin One');
```

```
INSERT INTO admins (id, admin_id, email, full_name)
```

```
VALUES (admin_id_seq.NEXTVAL, 'admin002', 'admin2@aiub.edu', 'Admin Two');
```

```
INSERT INTO admins (id, admin_id, email, full_name)
```

```
VALUES (admin_id_seq.NEXTVAL, 'admin003', 'admin3@aiub.edu', 'Admin Three');
```

```
INSERT INTO admins (id, admin_id, email, full_name)
```

```
VALUES (admin_id_seq.NEXTVAL, 'admin004', 'admin4@aiub.edu', 'Admin Four');
```

```
INSERT INTO admins (id, admin_id, email, full_name)
```

```
VALUES (admin_id_seq.NEXTVAL, 'admin005', 'admin5@aiub.edu', 'Admin Five');
```

The screenshot displays the Oracle Database Express Edition web interface. The top navigation bar includes 'Home', 'Logout', and 'Help' links. The user is logged in as 'WEBUSER'. The main area shows a list of SQL commands that have been executed, each preceded by a timestamp and the word 'ago'. The commands are all 'INSERT INTO' statements for the 'admins' table, using 'admin_id_seq.NEXTVAL' for the primary key and providing email and full name for five different administrators. Below the list of commands, there is a 'Results' tab and a 'History' tab. The 'Results' tab is currently selected, showing a table with columns 'Time', 'SQL', and 'Schema'. The table contains five rows of data, each corresponding to one of the 'INSERT' statements. The 'Schema' column for all rows is 'WEBUSER'. The bottom of the screenshot shows the Windows taskbar with the date and time '1:16 AM 12/15/2023'.

```
ORACLE Database Express Edition
```

User: WEBUSER

Home Logout Help

Home > SQL > SQL Commands

☒ Autocommit Display 10 Save Run

```
INSERT INTO admins (id, admin_id, email, full_name)
/VALUES (admin_id_seq.NEXTVAL, 'admin001', 'admin1@aiub.edu', 'Admin One');

INSERT INTO admins (id, admin_id, email, full_name)
/VALUES (admin_id_seq.NEXTVAL, 'admin002', 'admin2@aiub.edu', 'Admin Two');

INSERT INTO admins (id, admin_id, email, full_name)
/VALUES (admin_id_seq.NEXTVAL, 'admin003', 'admin3@aiub.edu', 'Admin Three');

INSERT INTO admins (id, admin_id, email, full_name)
/VALUES (admin_id_seq.NEXTVAL, 'admin004', 'admin4@aiub.edu', 'Admin Four');

INSERT INTO admins (id, admin_id, email, full_name)
/VALUES (admin_id_seq.NEXTVAL, 'admin005', 'admin5@aiub.edu', 'Admin Five');
```

Results Explain Describe Saved SQL History

Find Go

Time	SQL	Schema
31 seconds ago	INSERT INTO admins (id, admin_id, email, full_name) VALUES (admin_id_seq.NEXTVAL, 'admin001', 'admin1@aiub.edu', 'Admin One');	WEBUSER
61 seconds ago	INSERT INTO admins (id, admin_id, email, full_name) VALUES (admin_id_seq.NEXTVAL, 'admin002', 'admin2@aiub.edu', 'Admin Two');	WEBUSER
2 minutes ago	INSERT INTO admins (id, admin_id, email, full_name) VALUES (admin_id_seq.NEXTVAL, 'admin003', 'admin3@aiub.edu', 'Admin Three');	WEBUSER
2 minutes ago	INSERT INTO admins (id, admin_id, email, full_name) VALUES (admin_id_seq.NEXTVAL, 'admin004', 'admin4@aiub.edu', 'Admin Four');	WEBUSER
2 minutes ago	INSERT INTO admins (id, admin_id, email, full_name) VALUES (admin_id_seq.NEXTVAL, 'admin005', 'admin5@aiub.edu', 'Admin Five');	WEBUSER
3 minutes ago	INSERT INTO admins (id, admin_id, email, full_name) VALUES (admin_id_seq.NEXTVAL, 'admin006', 'admin6@aiub.edu', 'Admin Six');	WEBUSER
3 minutes ago	INSERT INTO admins (id, admin_id, email, full_name) VALUES (admin_id_seq.NEXTVAL, 'admin007', 'admin7@aiub.edu', 'Admin Seven');	WEBUSER
4 minutes ago	INSERT INTO admins (id, admin_id, email, full_name) VALUES (admin_id_seq.NEXTVAL, 'admin008', 'admin8@aiub.edu', 'Admin Eight');	WEBUSER
6 minutes ago	INSERT INTO users (id, student_id, email, full_name, gender, phone_number, program_level, department)	WEBUSER
7 minutes ago	INSERT INTO users (id, student_id, email, full_name, gender, phone_number, program_level, department)	WEBUSER

1:16 AM 12/15/2023

Insert Data into Tournaments table

```
INSERT INTO tournaments (id, title, photo_url, registration_deadline, status, created_by)
```

```
VALUES (tournament_id_seq.NEXTVAL, 'AIUB Annual Sports Festival 2025',  
'https://example.com/images/sports_fest_2025.jpg', TO_DATE('2025-02-15 23:59:59', 'YYYY-MM-DD HH24:MI:SS'), 'ACTIVE', 1);
```

```
INSERT INTO tournaments (id, title, photo_url, registration_deadline, status, created_by)
```

```
VALUES (tournament_id_seq.NEXTVAL, 'Inter-Department Cricket Championship',  
'https://example.com/images/cricket_champ.jpg', TO_DATE('2025-01-20 23:59:59', 'YYYY-MM-DD HH24:MI:SS'), 'ACTIVE', 2);
```

```
INSERT INTO tournaments (id, title, photo_url, registration_deadline, status, created_by)
```

```
VALUES (tournament_id_seq.NEXTVAL, 'AIUB Basketball Tournament',  
'https://example.com/images/basketball_tour.jpg', TO_DATE('2025-02-10 23:59:59', 'YYYY-MM-DD HH24:MI:SS'), 'CLOSED', 1);
```

```
INSERT INTO tournaments (id, title, photo_url, registration_deadline, status, created_by)
```

```
VALUES (tournament_id_seq.NEXTVAL, 'AIUB Football League',  
'https://example.com/images/football_league.jpg', TO_DATE('2025-01-25 23:59:59', 'YYYY-MM-DD HH24:MI:SS'), 'ACTIVE', 3);
```

```
INSERT INTO tournaments (id, title, photo_url, registration_deadline, status, created_by)
```

```
VALUES (tournament_id_seq.NEXTVAL, 'AIUB Chess Championship',  
'https://example.com/images/chess_champ.jpg', TO_DATE('2025-02-05 23:59:59', 'YYYY-MM-DD HH24:MI:SS'), 'COMPLETED', 4);
```

The screenshot displays the Oracle Database Express Edition web interface. The top navigation bar includes the Oracle logo, 'Database Express Edition', and user information 'User: AUB_PORTAL'. The main content area is titled 'SQL Commands' and contains a text editor with the following SQL code:

```
-- 'ACTIVE',  
3  
);  
  
INSERT INTO tournaments (  
id, title, photo_url, registration_deadline, status, created_by  
)  
VALUES (  
tournament_id_seq.NEXTVAL,  
'AIUB Chess Championship',  
'https://example.com/images/chess_champ.jpg',  
TO_DATE('2025-02-05 23:59:59', 'YYYY-MM-DD HH24:MI:SS'),  
'COMPLETED',  
4  
);  
  
select * from tournaments ;
```

Below the editor, the 'Results' tab is active, showing a table with 5 rows of tournament data:

ID	TITLE	PHOTO_URL	REGISTRATION_DEADLINE	STATUS	CREATED_BY	CREATED_AT
3	Inter-Department Cricket Championship	https://example.com/images/cricket_champ.jpg	20-JAN-25 11:59:59 000000 PM	ACTIVE	2	14-DEC-25 08:26:53 214000 PM
4	AIUB Basketball Tournament	https://example.com/images/basketball_tour.jpg	10-FEB-25 11:59:59 000000 PM	CLOSED	1	14-DEC-25 08:27:53 568000 PM
6	AIUB Chess Championship	https://example.com/images/chess_champ.jpg	05-FEB-25 11:59:59 000000 PM	COMPLETED	4	14-DEC-25 08:27:26 130000 PM
2	AIUB Annual Sports Festival 2025	https://example.com/images/sports_fest_2025.jpg	15-FEB-25 11:59:59 000000 PM	ACTIVE	1	14-DEC-25 08:26:29 111000 PM
5	AIUB Football League	https://example.com/images/football_league.jpg	25-JAN-25 11:59:59 000000 PM	ACTIVE	3	14-DEC-25 08:27:13 707000 PM

At the bottom of the results section, it states '5 rows returned in 0.00 seconds'. The footer of the interface shows 'Language: en-us' and 'Application Express 2.1.0.00.39 Copyright © 1999, 2005, Oracle. All rights reserved.'

Insert Data into Tournament_games table

```
INSERT INTO tournament_games (id, tournament_id, category, game_name, game_type, fee_per_person)
```

```
VALUES (game_id_seq.NEXTVAL, 1, 'Male', 'Cricket', 'Custom', 500);
```

```
INSERT INTO tournament_games (id, tournament_id, category, game_name, game_type, fee_per_person)
```

```
VALUES (game_id_seq.NEXTVAL, 1, 'Female', 'Badminton', 'Solo', 300);
```

```
INSERT INTO tournament_games (id, tournament_id, category, game_name, game_type, fee_per_person)
```

```
VALUES (game_id_seq.NEXTVAL, 1, 'Mix', 'Volleyball', 'Custom', 400);
```

```
INSERT INTO tournament_games (id, tournament_id, category, game_name, game_type, fee_per_person)
```

```
VALUES (game_id_seq.NEXTVAL, 2, 'Male', 'Cricket', 'Custom', 600);
```

```
INSERT INTO tournament_games (id, tournament_id, category, game_name, game_type, fee_per_person)
```

```
VALUES (game_id_seq.NEXTVAL, 3, 'Mix', 'Basketball', 'Custom', 450);
```

The screenshot shows the Oracle Database Express Edition interface. The SQL Command window contains the following commands:

```
VALUES (game_id_seq.NEXTVAL, 2, 'Female', 'Badminton', 'Solo', 300);
INSERT INTO tournament_games (
  id, tournament_id, category, game_name, game_type, fee_per_person
)
VALUES (game_id_seq.NEXTVAL, 2, 'Mix', 'Volleyball', 'Custom', 400);
INSERT INTO tournament_games (
  id, tournament_id, category, game_name, game_type, fee_per_person
)
VALUES (game_id_seq.NEXTVAL, 3, 'Male', 'Cricket', 'Custom', 600);
INSERT INTO tournament_games (
  id, tournament_id, category, game_name, game_type, fee_per_person
)
VALUES (game_id_seq.NEXTVAL, 4, 'Mix', 'Basketball', 'Custom', 450);
select * from tournament_games;
```

The Results window shows the following data:

ID	TOURNAMENT_ID	CATEGORY	GAME_NAME	GAME_TYPE	FEE_PER_PERSON	CREATED_AT
3	2	Female	Badminton	Solo	300	14-DEC-25 08:35:11.492000 PM
4	2	Mix	Volleyball	Custom	400	14-DEC-25 08:35:18.015000 PM
6	4	Mix	Basketball	Custom	450	14-DEC-25 08:35:38.549000 PM
2	2	Male	Cricket	Custom	500	14-DEC-25 08:35:02.505000 PM
5	3	Male	Cricket	Custom	600	14-DEC-25 08:35:29.010000 PM

5 rows returned in 0.00 seconds

Insert Data into Game_registrations table

```
INSERT INTO game_registrations (id, game_id, user_id, payment_status)
VALUES (registration_id_seq.NEXTVAL, 1, 1, 'PAID');

INSERT INTO game_registrations (id, game_id, user_id, payment_status)
VALUES (registration_id_seq.NEXTVAL, 2, 2, 'PENDING');

INSERT INTO game_registrations (id, game_id, user_id, payment_status)
VALUES (registration_id_seq.NEXTVAL, 3, 3, 'PAID');

INSERT INTO game_registrations (id, game_id, user_id, payment_status)
VALUES (registration_id_seq.NEXTVAL, 4, 4, 'FAILED');

INSERT INTO game_registrations (id, game_id, user_id, payment_status)
VALUES (registration_id_seq.NEXTVAL, 5, 5, 'PAID');

COMMIT;
```

ORACLE Database Express Edition

User: AUB_PORTAL

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```
INSERT INTO game_registrations (id, game_id, user_id, payment_status)
VALUES (registration_id_seq.NEXTVAL, 1, 1, 'PAID');

INSERT INTO game_registrations (id, game_id, user_id, payment_status)
VALUES (registration_id_seq.NEXTVAL, 2, 2, 'PENDING');

INSERT INTO game_registrations (id, game_id, user_id, payment_status)
VALUES (registration_id_seq.NEXTVAL, 3, 3, 'PAID');

INSERT INTO game_registrations (id, game_id, user_id, payment_status)
VALUES (registration_id_seq.NEXTVAL, 4, 4, 'FAILED');

INSERT INTO game_registrations (id, game_id, user_id, payment_status)
VALUES (registration_id_seq.NEXTVAL, 5, 5, 'PAID');

INSERT INTO game_registrations (id, game_id, user_id, payment_status)
VALUES (registration_id_seq.NEXTVAL, 3, 2, 'PAID');
```

Results Explain Describe Saved SQL History

ID	GAME_ID	USER_ID	REGISTRATION_DATE	PAYMENT_STATUS
2	2	2	14-DEC-25 08:52:38.934000 PM	PENDING
4	4	4	14-DEC-25 08:52:56.399000 PM	FAILED
3	3	3	14-DEC-25 08:52:46.110000 PM	PAID
5	5	5	14-DEC-25 08:53:01.950000 PM	PAID
6	3	2	14-DEC-25 08:54:05.060000 PM	PAID

5 rows returned in 0.00 seconds CSV Export

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

Windows taskbar showing search, task view, and various application icons. System clock: 2:54 AM, 12/15/2025.

Basic PL/SQL

2 Variables

1. DECLARE

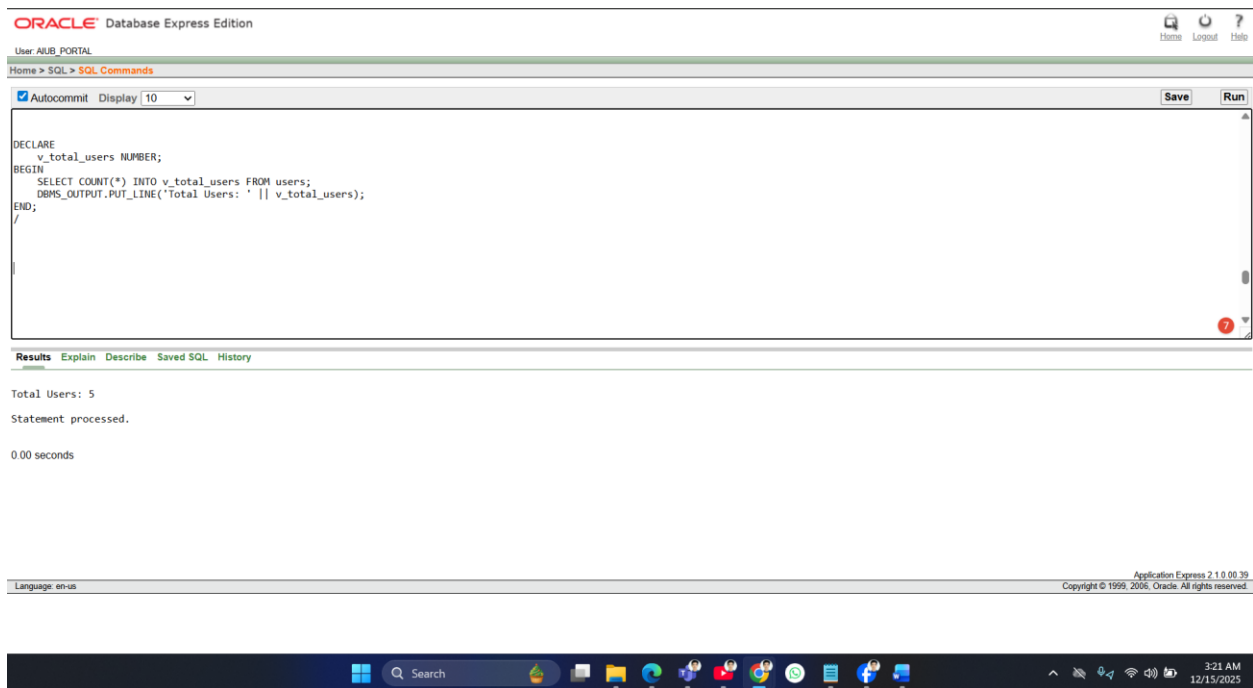
```
v_total_users NUMBER;
```

```
BEGIN
```

```
SELECT COUNT(*) INTO v_total_users FROM users;
```

```
DBMS_OUTPUT.PUT_LINE('Total Users: ' || v_total_users);
```

```
END;
```



2. DECLARE

```
v_first_user_name VARCHAR2(100);
```

```
BEGIN
```

```
SELECT full_name INTO v_first_user_name FROM users WHERE id = 1;
```

```
DBMS_OUTPUT.PUT_LINE('First User Name: ' || v_first_user_name);
```

```
END;
```

```
/
```

ORACLE Database Express Edition

User AUB_PORTAL

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```
DECLARE
  v_first_user_name VARCHAR2(100);
BEGIN
  SELECT full_name INTO v_first_user_name FROM users WHERE id = 1;
  DBMS_OUTPUT.PUT_LINE('First User Name: ' || v_first_user_name);
END;
```

Results Explain Describe Saved SQL History

First User Name: John Smith
Statement processed.
0.00 seconds

Language: en-us

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

2 Operators

1.DECLARE

v_sum NUMBER;

BEGIN

v_sum := 5 + 10;

DBMS_OUTPUT.PUT_LINE('5 + 10 = ' || v_sum);

END;

ORACLE Database Express Edition

User AUB_PORTAL

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```
DECLARE
  v_sum NUMBER;
BEGIN
  v_sum := 5 + 10;
  DBMS_OUTPUT.PUT_LINE('5 + 10 = ' || v_sum);
END;
```

Results Explain Describe Saved SQL History

5 + 10 = 15
Statement processed.
0.00 seconds

Language: en-us

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

2.DECLARE

```
v_count NUMBER;
```

```
BEGIN
```

```
SELECT COUNT(*) INTO v_count FROM users;
```

```
IF v_count > 3 THEN
```

```
    DBMS_OUTPUT.PUT_LINE('More than 3 users.');
```

```
ELSE
```

```
    DBMS_OUTPUT.PUT_LINE('3 or fewer users.');
```

```
END IF;
```

```
END;
```

The screenshot displays the Oracle Database Express Edition web interface. At the top, it shows 'ORACLE Database Express Edition' and 'User: AUB PORTAL'. Below this is a navigation bar with 'Home', 'Logout', and 'Help' links. The main area is titled 'Home > SQL > SQL Commands' and contains a text editor with the following SQL script:

```
DECLARE
  v_count NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_count FROM users;
  IF v_count > 3 THEN
    DBMS_OUTPUT.PUT_LINE('More than 3 users.');
```

The script is executed, and the results are displayed in the 'Results' tab. The output shows 'More than 3 users.' followed by 'Statement processed.' and '0.00 seconds'.

At the bottom of the interface, there is a footer with 'Language: en-us' and 'Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.'

2 Single-row function

1. DECLARE

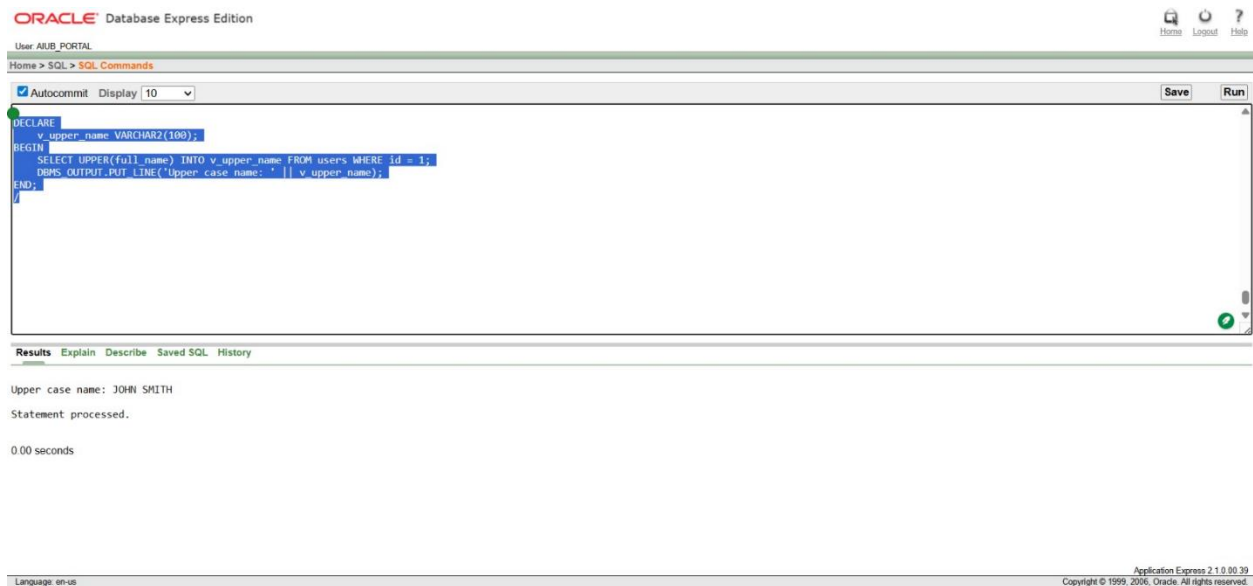
```
v_upper_name VARCHAR2(100);
```

```
BEGIN
```

```
SELECT UPPER(full_name) INTO v_upper_name FROM users WHERE id = 1;
```

```
DBMS_OUTPUT.PUT_LINE('Upper case name: ' || v_upper_name);
```

```
END;
```



2. DECLARE

```
v_name_length NUMBER;
```

```
BEGIN
```

```
SELECT LENGTH(full_name) INTO v_name_length FROM users WHERE id = 1;
```

```
DBMS_OUTPUT.PUT_LINE('Name length: ' || v_name_length);
```

```
END;
```

User: AUB_PORTAL

Home > SQL > SQL Commands

```
Autocommit Display 10 Save Run
```

```
DECLARE
  v_name_length NUMBER;
BEGIN
  SELECT LENGTH(full_name) INTO v_name_length FROM users WHERE id = 1;
  DBMS_OUTPUT.PUT_LINE('Name length: ' || v_name_length);
END;
```

[Results](#) [Explain](#) [Describe](#) [Saved SQL](#) [History](#)

Name length: 10

Statement processed.

0.02 seconds

Application Express 2.1.0.00.39

Language: en-us

Copyright © 1999, 2006, Oracle. All rights reserved.

2 group function

1. DECLARE

```
v_count NUMBER;
```

```
BEGIN
```

```
  SELECT COUNT(*) INTO v_count FROM users;
```

```
  DBMS_OUTPUT.PUT_LINE('Total Users: ' || v_count);
```

```
END;
```

User: AUB_PORTAL

Home > SQL > SQL Commands

```
Autocommit Display 10 Save Run
```

```
DECLARE
  v_count NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_count FROM users;
  DBMS_OUTPUT.PUT_LINE('Total Users: ' || v_count);
END;
```

[Results](#) [Explain](#) [Describe](#) [Saved SQL](#) [History](#)

Total Users: 5

Statement processed.

0.00 seconds

Application Express 2.1.0.00.39

Language: en-us

Copyright © 1999, 2006, Oracle. All rights reserved.

2. DECLARE

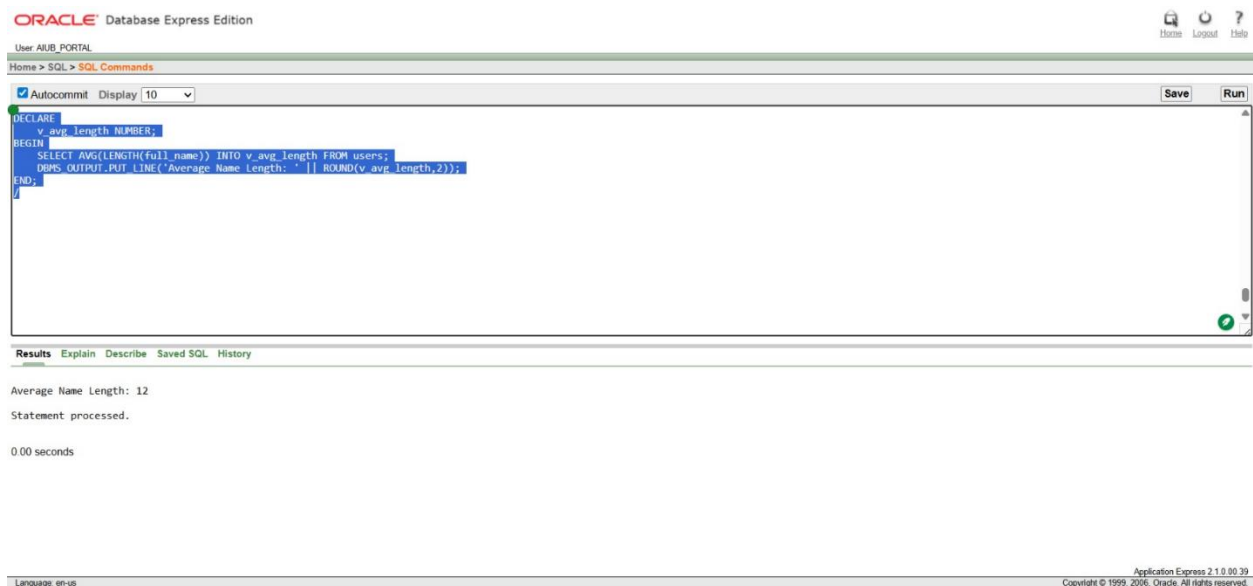
```
v_avg_length NUMBER;
```

```
BEGIN
```

```
SELECT AVG(LENGTH(full_name)) INTO v_avg_length FROM users;
```

```
DBMS_OUTPUT.PUT_LINE('Average Name Length: ' || ROUND(v_avg_length,2));
```

```
END;
```



2 conditional statements

1. DECLARE

```
v_count NUMBER;
```

```
BEGIN
```

```
SELECT COUNT(*) INTO v_count FROM users;
```

```
IF v_count > 3 THEN
```

```
    DBMS_OUTPUT.PUT_LINE('More than 3 users.');
```

```
ELSE
```

```
    DBMS_OUTPUT.PUT_LINE('3 or fewer users.');
```

```
END IF; END;
```

ORACLE Database Express Edition

User AUB_PORTAL

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```
DECLARE
  v_count NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_count FROM users;
  IF v_count > 3 THEN
    DBMS_OUTPUT.PUT_LINE('More than 3 users. ');
  ELSE
    DBMS_OUTPUT.PUT_LINE('3 or fewer users. ');
  END IF;
END;
```

Results Explain Describe Saved SQL History

More than 3 users.
Statement processed.
0.02 seconds

Language: en-us

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

2.BEGIN

```
FOR rec IN (SELECT full_name, program_level FROM users) LOOP
  CASE rec.program_level
    WHEN 'Undergraduate' THEN DBMS_OUTPUT.PUT_LINE(rec.full_name || ' is Undergraduate');
    WHEN 'Postgraduate' THEN DBMS_OUTPUT.PUT_LINE(rec.full_name || ' is Postgraduate');
  END CASE;
END LOOP;
END;
```

ORACLE Database Express Edition

User AUB_PORTAL

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```
BEGIN
  FOR rec IN (SELECT full_name, program_level FROM users) LOOP
    CASE rec.program_level
      WHEN 'Undergraduate' THEN DBMS_OUTPUT.PUT_LINE(rec.full_name || ' is Undergraduate');
      WHEN 'Postgraduate' THEN DBMS_OUTPUT.PUT_LINE(rec.full_name || ' is Postgraduate');
    END CASE;
  END LOOP;
END;
```

Results Explain Describe Saved SQL History

John Smith is Undergraduate
Sarah Johnson is Undergraduate
Robert Davis is Postgraduate
Emily Brown is Undergraduate
Michael Wilson is Postgraduate
Statement processed.
0.00 seconds

Language: en-us

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

2 Subquery

DECLARE

 v_dept_count NUMBER;

BEGIN

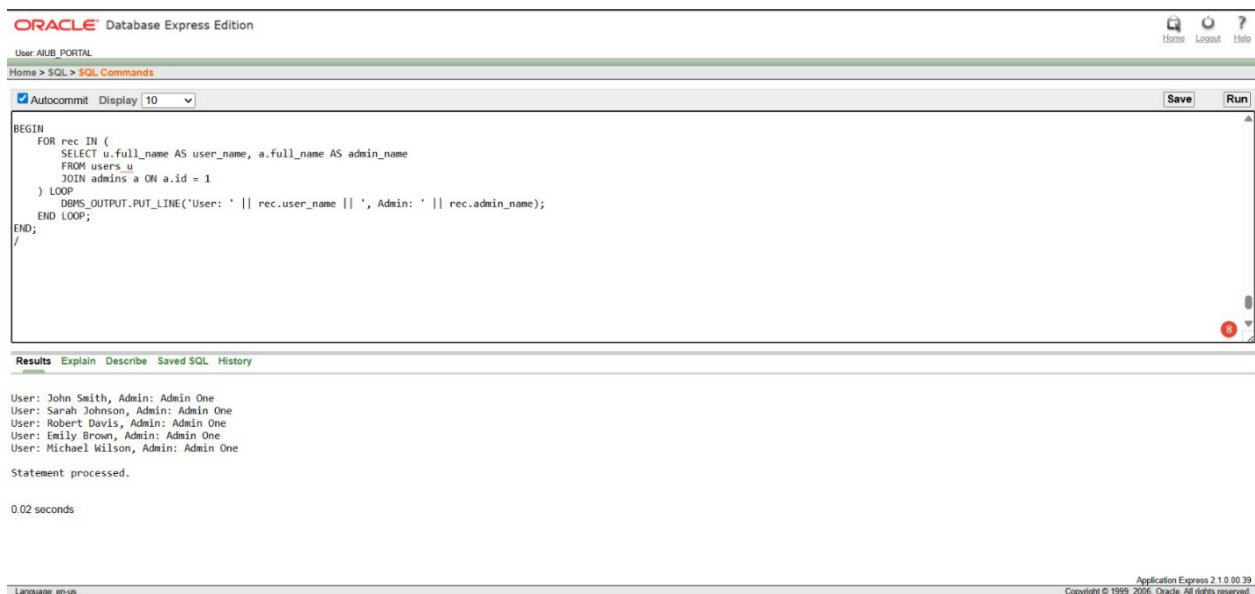
 SELECT COUNT(*) INTO v_dept_count

 FROM users

 WHERE department = (SELECT department FROM users WHERE id = 1);

 DBMS_OUTPUT.PUT_LINE('Users in same department as user 1: ' || v_dept_count);

END;



The screenshot shows the Oracle Database Express Edition web interface. The top navigation bar includes 'Home', 'Logout', and 'Help' links. The main content area displays the SQL command window with the following code:

```
BEGIN
  FOR rec IN (
    SELECT u.full_name AS user_name, a.full_name AS admin_name
    FROM users u
    JOIN admins a ON a.id = 1
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('User: ' || rec.user_name || ', Admin: ' || rec.admin_name);
  END LOOP;
END;
```

Below the code window, the 'Results' tab is active, showing the output of the SQL statement:

```
User: John Smith, Admin: Admin One
User: Sarah Johnson, Admin: Admin One
User: Robert Davis, Admin: Admin One
User: Emily Brown, Admin: Admin One
User: Michael Wilson, Admin: Admin One
Statement processed.

0.02 seconds
```

The bottom status bar indicates the language is 'en-us' and the application is 'Express 2.1.0.80.39'.

2.DECLARE

 v_email VARCHAR2(100);

BEGIN

 SELECT email INTO v_email

 FROM users

 WHERE id = (SELECT MAX(id) FROM users);

 DBMS_OUTPUT.PUT_LINE('Email of user with max ID: ' || v_email);

END;

Oracle Database Express Edition interface. The SQL Command window contains the following code:

```
DECLARE
  v_email VARCHAR2(100);
BEGIN
  SELECT email INTO v_email
  FROM users
  WHERE id = (SELECT MAX(id) FROM users);
  DBMS_OUTPUT.PUT_LINE('Email of user with max ID: ' || v_email);
END;
```

The Results pane shows the output: "Email of user with max ID: 20-12345-5@student.aiub.edu". The statement was processed successfully in 0.00 seconds.

2 Joining

1. BEGIN

```
FOR rec IN (
  SELECT u.full_name AS user_name, a.full_name AS admin_name
  FROM users u
  JOIN admins a ON a.id = 1 ) LOOP
  DBMS_OUTPUT.PUT_LINE('User: ' || rec.user_name || ', Admin: ' || rec.admin_name);
END LOOP;
```

END;

Oracle Database Express Edition interface. The SQL Command window contains the following code:

```
BEGIN
  FOR rec IN (
    SELECT u.full_name AS user_name, a.full_name AS admin_name
    FROM users u
    JOIN admins a ON a.id = 1
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('User: ' || rec.user_name || ', Admin: ' || rec.admin_name);
  END LOOP;
END;
```

The Results pane shows the output:

```
User: John Smith, Admin: Admin One
User: Sarah Johnson, Admin: Admin One
User: Robert Davis, Admin: Admin One
User: Emily Brown, Admin: Admin One
User: Michael Wilson, Admin: Admin One
```

The statement was processed successfully in 0.02 seconds.

2. BEGIN

```
FOR rec IN (  
  
    SELECT u.full_name AS user_name, a.full_name AS admin_name  
  
    FROM users u, admins a  
  
    WHERE a.id = 2 AND u.id <= 3  
  
) LOOP  
  
    DBMS_OUTPUT.PUT_LINE('User: ' || rec.user_name || ', Admin: ' || rec.admin_name);  
  
END LOOP;  
  
END;
```

The screenshot displays the Oracle Database Express Edition web interface. The top navigation bar includes 'Home', 'Logout', and 'Help' links. The main content area shows the 'SQL Commands' tab with a text editor containing the SQL code from the previous block. The 'Autocommit' checkbox is checked, and the 'Display' is set to 10. Below the editor, the 'Results' tab is active, showing the output of the SQL statement: 'User: John Smith, Admin: Admin Two', 'User: Sarah Johnson, Admin: Admin Two', and 'User: Robert Davis, Admin: Admin Two'. The status 'Statement processed.' and execution time '0.00 seconds' are also visible. The bottom status bar indicates 'Language: en-us' and 'Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.'

```
ORACLE Database Express Edition
```

User: AUB_PORTAL

Home > SQL > SQL Commands

☒ Autocommit Display: 10 Save Run

```
BEGIN  
  FOR rec IN (  
    SELECT u.full_name AS user_name, a.full_name AS admin_name  
    FROM users u, admins a  
    WHERE a.id = 2 AND u.id <= 3  
  ) LOOP  
    DBMS_OUTPUT.PUT_LINE('User: ' || rec.user_name || ', Admin: ' || rec.admin_name);  
  END LOOP;  
END;  
/
```

Results Explain Describe Saved SQL History

User: John Smith, Admin: Admin Two
User: Sarah Johnson, Admin: Admin Two
User: Robert Davis, Admin: Admin Two

Statement processed.

0.00 seconds

Language: en-us Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.

Advance PL/SQL

2 Stored functions

Function 1: Get total users

```
CREATE OR REPLACE FUNCTION get_total_users RETURN NUMBER IS
```

```
    v_count NUMBER;
```

```
BEGIN
```

```
    SELECT COUNT(*) INTO v_count FROM users;
```

```
    RETURN v_count;
```

```
END;
```

```
/
```

Test Function 1

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('Total Users: ' || get_total_users);
```

```
END;
```

```
/
```

The screenshot displays the Oracle Database Express Edition web interface. The top navigation bar includes 'ORACLE Database Express Edition', 'User: AUB_PORTAL', and links for 'Home', 'Logout', and 'Help'. The breadcrumb trail shows 'Home > SQL > SQL Commands'. The main editor area contains the PL/SQL code for the function and its test. The code is as follows:

```
CREATE OR REPLACE FUNCTION get_total_users RETURN NUMBER IS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM users;
    RETURN v_count;
END;
/
BEGIN
    DBMS_OUTPUT.PUT_LINE('Total Users: ' || get_total_users);
END;
/
```

Below the editor, the 'Results' tab is active, showing the output of the execution:

```
Total Users: 5
Statement processed.
0.00 seconds
```

The footer of the interface includes 'Language: en-us' and 'Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.'

Function 2: Get email by student_id

```
CREATE OR REPLACE FUNCTION get_email(p_student_id VARCHAR2) RETURN  
VARCHAR2 IS
```

```
    v_email VARCHAR2(100);
```

```
BEGIN
```

```
    SELECT email INTO v_email FROM users WHERE student_id = p_student_id;
```

```
    RETURN v_email;
```

```
END;
```

```
/
```

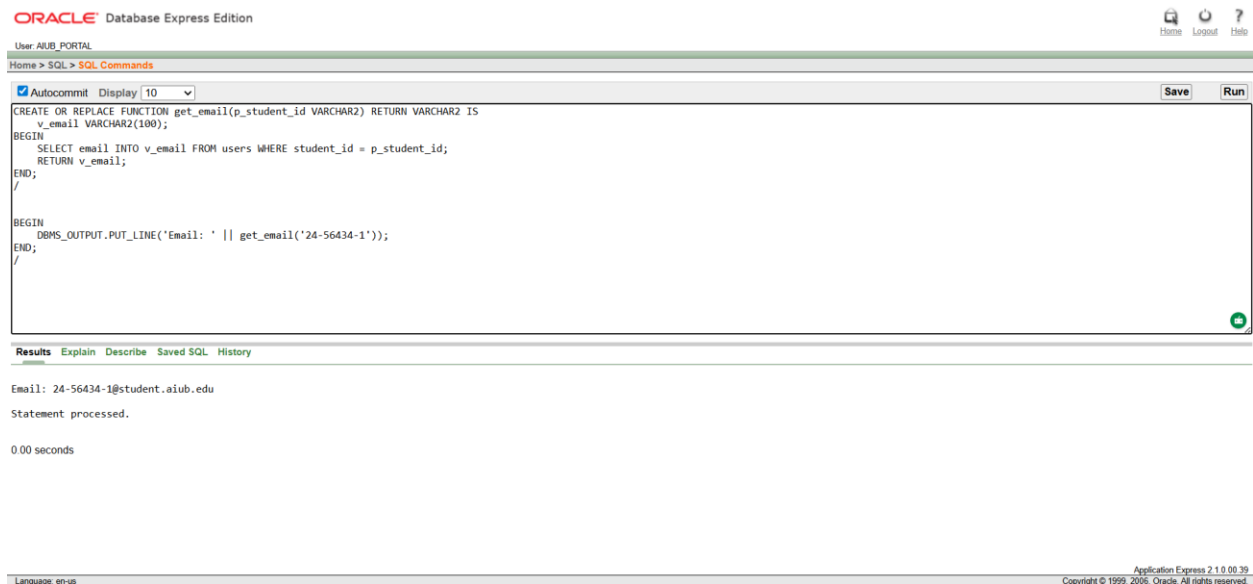
Test Function 2

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('Email: ' || get_email('24-56434-1'));
```

```
END;
```

```
/
```



2. Stored Procedures

Procedure 1: Print all usernames

```
CREATE OR REPLACE PROCEDURE print_all_users IS
```

```
BEGIN
```

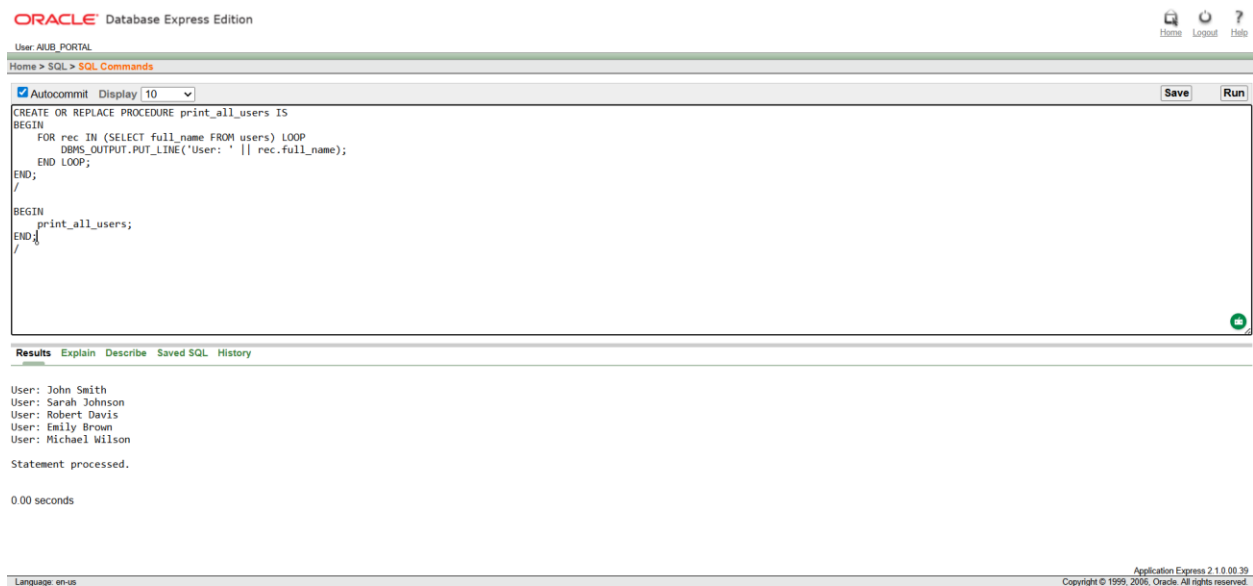
```
    FOR rec IN (SELECT full_name FROM users) LOOP
```

```

        DBMS_OUTPUT.PUT_LINE('User: ' || rec.full_name);
    END LOOP;
END;
/

-- Test Procedure 1
BEGIN
    print_all_users;
END;
/

```



Procedure 2: Update name_edit_count for a student

```

CREATE OR REPLACE PROCEDURE update_name_edit(p_id NUMBER) IS
BEGIN
    UPDATE users SET name_edit_count = name_edit_count + 1 WHERE id = p_id;
    COMMIT;
END;

Test Procedure 2
BEGIN
    update_name_edit(1); END;

```

ORACLE Database Express Edition

User: AUB_PORTAL

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```
CREATE OR REPLACE PROCEDURE update_name_edit(p_id NUMBER) IS
BEGIN
    UPDATE users SET name_edit_count = name_edit_count + 1 WHERE id = p_id;
    COMMIT;
END;

BEGIN
    update_name_edit(1);
END;
/
```

Results Explain Describe Saved SQL History

Statement processed.

0.01 seconds

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

2 Table-Based Record

Example 1: Using %ROWTYPE

DECLARE

v_user users%ROWTYPE;

BEGIN

SELECT * INTO v_user FROM users WHERE id = 1;

DBMS_OUTPUT.PUT_LINE('Name: ' || v_user.full_name || ', Email: ' || v_user.email);

END;

ORACLE Database Express Edition

User: AUB_PORTAL

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```
DECLARE
    v_user users%ROWTYPE;
BEGIN
    SELECT * INTO v_user FROM users WHERE id = 1;
    DBMS_OUTPUT.PUT_LINE('Name: ' || v_user.full_name || ', Email: ' || v_user.email);
END;
/
```

Results Explain Describe Saved SQL History

Name: John Smith, Email: 24-56434-1@student.aiub.edu

Statement processed.

0.00 seconds

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

Example 2: Table of records

DECLARE

```
TYPE t_users IS TABLE OF users%ROWTYPE;
```

```
v_users t_users;
```

BEGIN

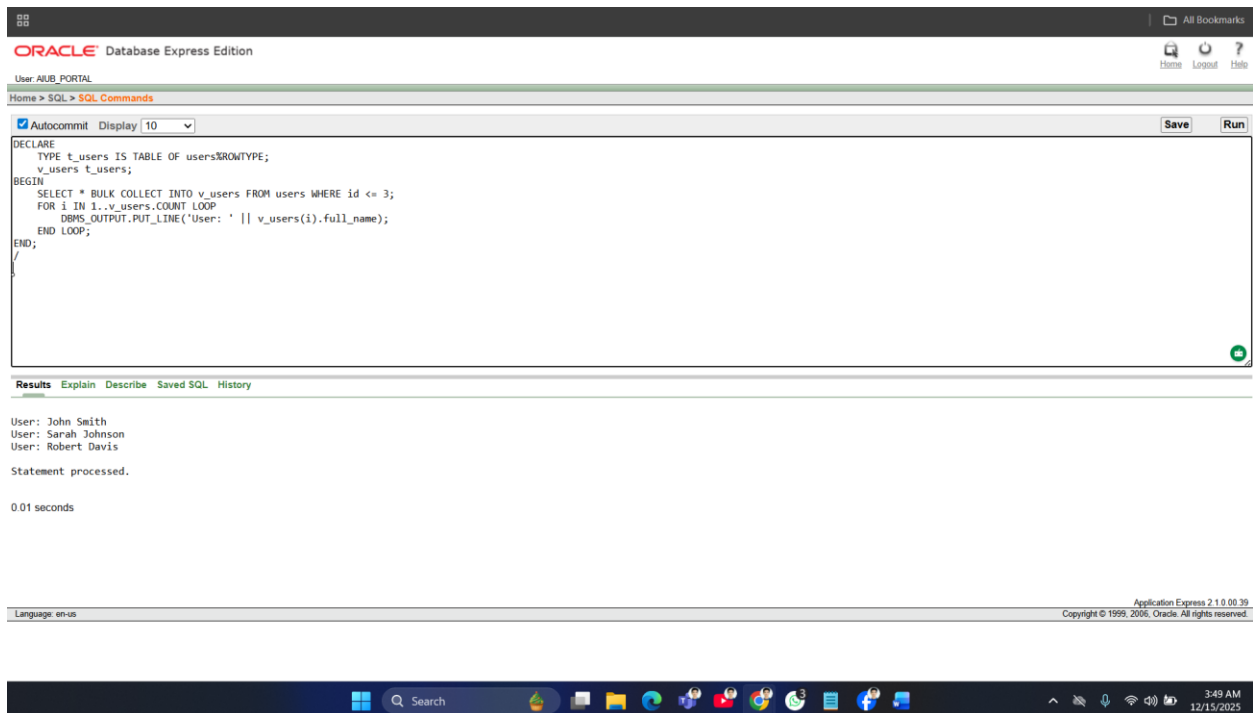
```
SELECT * BULK COLLECT INTO v_users FROM users WHERE id <= 3;
```

```
FOR i IN 1..v_users.COUNT LOOP
```

```
    DBMS_OUTPUT.PUT_LINE('User: ' || v_users(i).full_name);
```

```
END LOOP;
```

END;



2 Explicit Cursor

Cursor 1: Print all emails

DECLARE

```
CURSOR c_users IS SELECT email FROM users;
```

```
v_email users.email%TYPE;
```

BEGIN


```

OPEN c_users;

LOOP

    FETCH c_users INTO v_email;

    EXIT WHEN c_users%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('Email: ' || v_email);

END LOOP;

CLOSE c_users;

END;

/

```

The screenshot shows the Oracle Database Express Edition web interface. The top navigation bar includes 'Home', 'Logout', and 'Help' links. The main content area displays the SQL Commands window with the following code:

```

DECLARE
    CURSOR c_users IS SELECT email FROM users;
    v_email users.email%TYPE;
BEGIN
    OPEN c_users;
    LOOP
        FETCH c_users INTO v_email;
        EXIT WHEN c_users%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE('Email: ' || v_email);
    END LOOP;
    CLOSE c_users;
END;

```

Below the code editor, the 'Results' tab is active, showing the output of the SQL execution:

```

Email: 20-12345-5@student.aiub.edu
Email: 21-23456-4@student.aiub.edu
Email: 22-34567-3@student.aiub.edu
Email: 23-45123-2@student.aiub.edu
Email: 24-56434-1@student.aiub.edu

```

Below the results, the status 'Statement processed.' and the execution time '0.00 seconds' are displayed. The footer of the interface shows 'Application Express 2.1.0.00.39' and 'Copyright © 1999, 2006, Oracle. All rights reserved.'

Cursor 2: Count users by program_level

```

DECLARE

    CURSOR c_prog IS SELECT program_level, COUNT(*) AS cnt FROM users GROUP BY
    program_level;

    v_prog c_prog%ROWTYPE;

BEGIN

    OPEN c_prog;

    LOOP

```

```

    FETCH c_prog INTO v_prog;

    EXIT WHEN c_prog%NOTFOUND;

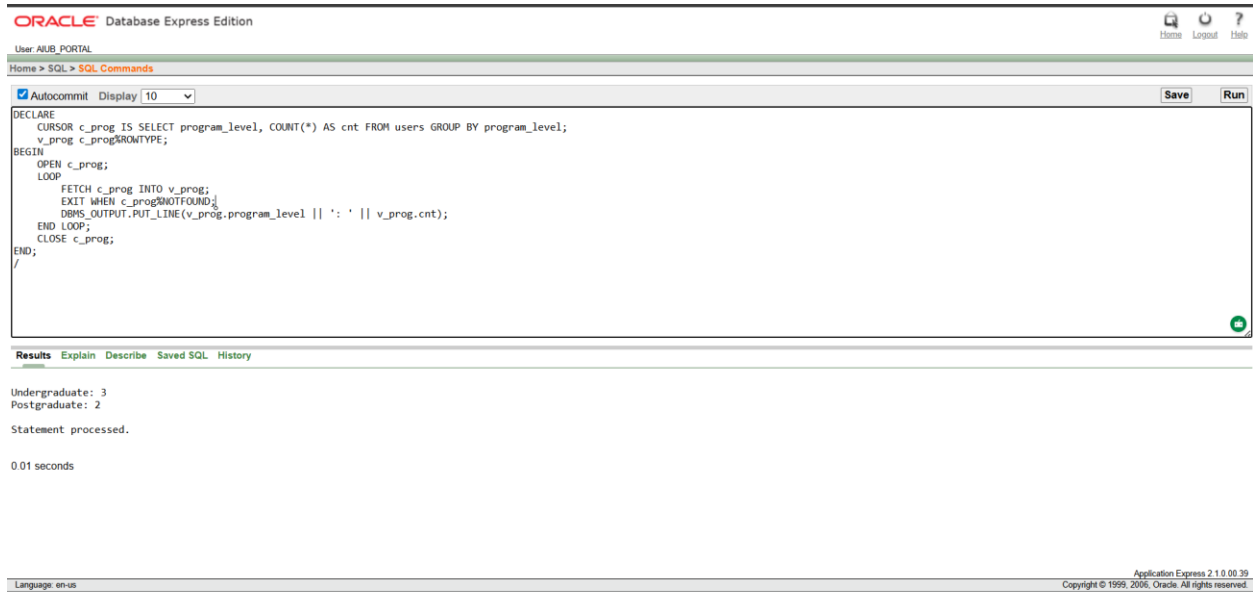
    DBMS_OUTPUT.PUT_LINE(v_prog.program_level || ': ' || v_prog.cnt);

END LOOP;

CLOSE c_prog;

END;

```



2 Cursor-Based Record

Example 1: Cursor with record

```

DECLARE

  CURSOR c_users IS SELECT full_name, email FROM users;

  v_user c_users%ROWTYPE;

BEGIN

  OPEN c_users;

  LOOP

    FETCH c_users INTO v_user;

    EXIT WHEN c_users%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('Name: ' || v_user.full_name || ', Email: ' || v_user.email);

```

```

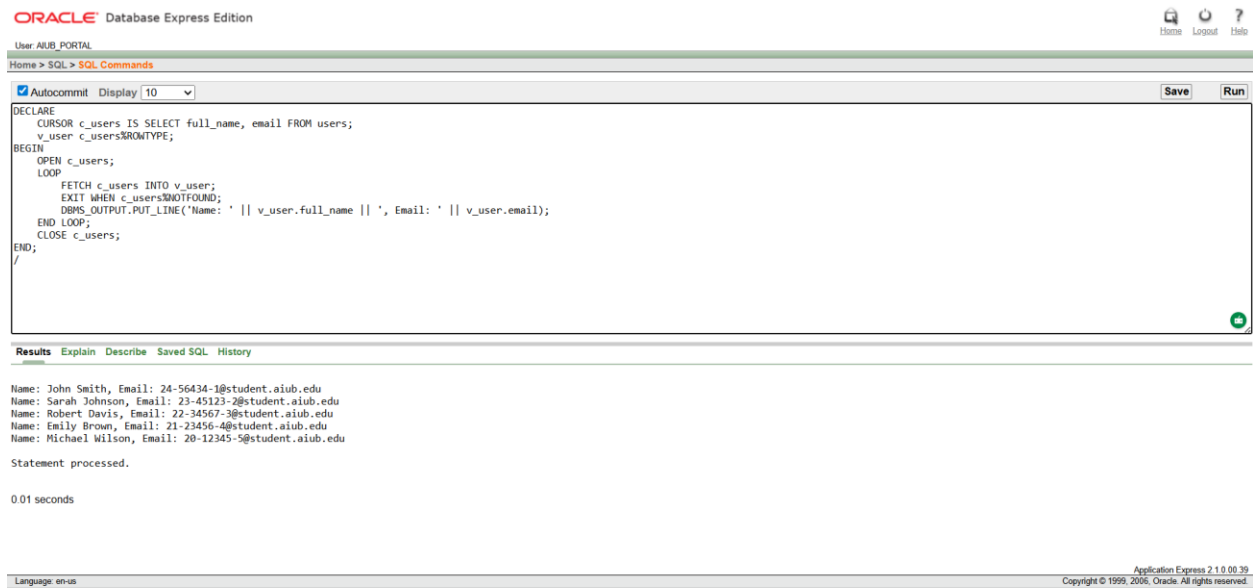
END LOOP;

CLOSE c_users;

END;

/

```



Example 2: Cursor with parameter

```

DECLARE

  CURSOR c_students(p_dept VARCHAR2) IS SELECT full_name FROM users WHERE
  department = p_dept;

  v_user c_students%ROWTYPE;

BEGIN

  OPEN c_students('CSE');

  LOOP

    FETCH c_students INTO v_user;

    EXIT WHEN c_students%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('CSE Student: ' || v_user.full_name);

  END LOOP;

  CLOSE c_students;

END;

```

ORACLE Database Express Edition

User AUB_PORTAL

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```

DECLARE
  CURSOR c_students(p_dept VARCHAR2) IS SELECT full_name FROM users WHERE department = p_dept;
  v_user c_students%ROWTYPE;
BEGIN
  OPEN c_students('CSE');
  LOOP
    FETCH c_students INTO v_user;
    EXIT WHEN c_students%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('CSE Student: ' || v_user.full_name);
  END LOOP;
  CLOSE c_students;
END;
/

```

Results Explain Describe Saved SQL History

CSE Student: John Smith
Statement processed.
0.02 seconds

Language: en-us

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

2 Row-Level Trigger

Trigger 1: Before insert, set profile_completed = 0

CREATE OR REPLACE TRIGGER trg_before_insert_users

BEFORE INSERT ON users

FOR EACH ROW

BEGIN

:NEW.profile_completed := 0;

END;

ORACLE Database Express Edition

User AUB_PORTAL

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```

CREATE OR REPLACE TRIGGER trg_before_insert_users
BEFORE INSERT ON users
FOR EACH ROW
BEGIN
  :NEW.profile_completed := 0;
END;
/

```

Results Explain Describe Saved SQL History

Trigger created.
0.03 seconds

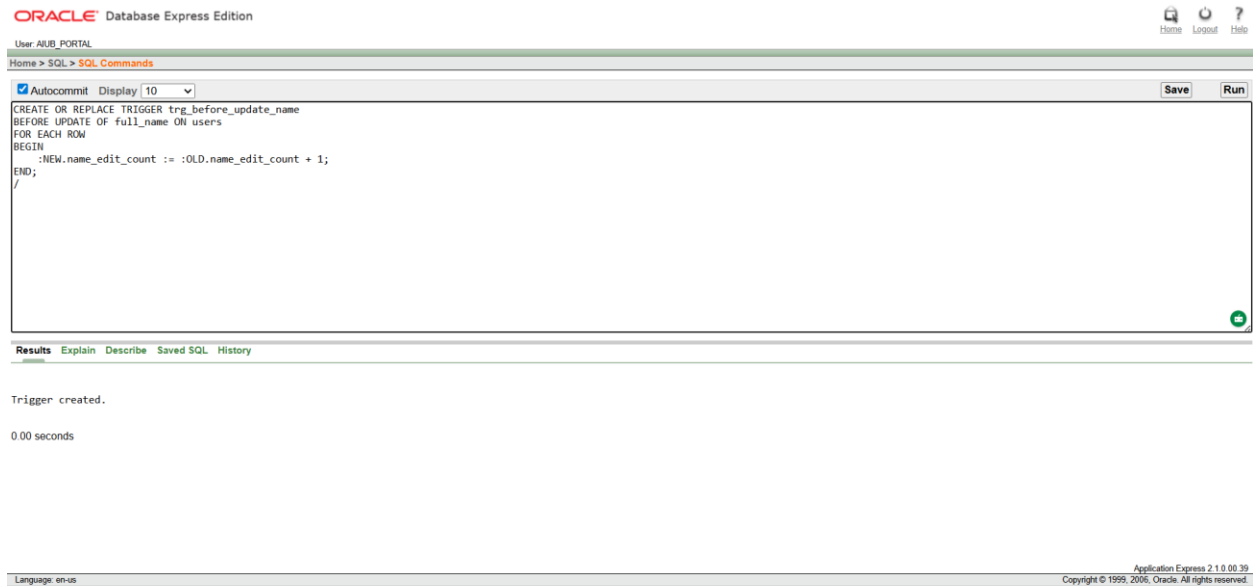
Language: en-us

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

```

CREATE OR REPLACE TRIGGER trg_before_update_name
BEFORE UPDATE OF full_name ON users
FOR EACH ROW
BEGIN
    :NEW.name_edit_count := :OLD.name_edit_count + 1;
END;

```



2Statement-Level Trigger

Trigger 1: After insert on users, log message

```

CREATE OR REPLACE TRIGGER trg_after_insert_users
AFTER INSERT ON users
BEGIN
    DBMS_OUTPUT.PUT_LINE('A new user has been inserted.');
```

END;

ORACLE Database Express Edition

User AUB_PORTAL

Home > SQL > SQL Commands

☒ Autocommit Display 10 Save Run

```
CREATE OR REPLACE TRIGGER trg_after_insert_users
AFTER INSERT ON users
BEGIN
    DBMS_OUTPUT.PUT_LINE('A new user has been inserted.');
```

END;

1
0

Results Explain Describe Saved SQL History

Trigger created.

0.00 seconds

Language: en-us

Application Express 2.1.0.05.39
Copyright © 1999, 2006, Oracle. All rights reserved.

Trigger 2: After delete on users, log message

CREATE OR REPLACE TRIGGER trg_after_delete_users

AFTER DELETE ON users

BEGIN

DBMS_OUTPUT.PUT_LINE('A user has been deleted.');

END;

ORACLE Database Express Edition

User AUB_PORTAL

Home > SQL > SQL Commands

☒ Autocommit Display 10 Save Run

```
CREATE OR REPLACE TRIGGER trg_after_delete_users
AFTER DELETE ON users
BEGIN
    DBMS_OUTPUT.PUT_LINE('A user has been deleted.');
```

END;

Results Explain Describe Saved SQL History

Trigger created.

0.01 seconds

Language: en-us

Application Express 2.1.0.05.39
Copyright © 1999, 2006, Oracle. All rights reserved.

2 Package

1.CREATE OR REPLACE PACKAGE pkg_users IS

 PROCEDURE print_user(p_id NUMBER);

 FUNCTION get_user_email(p_id NUMBER) RETURN VARCHAR2;

END pkg_users;

/

Package Body

CREATE OR REPLACE PACKAGE BODY pkg_users IS

 PROCEDURE print_user(p_id NUMBER) IS

 v_name VARCHAR2(100);

 BEGIN

 SELECT full_name INTO v_name FROM users WHERE id = p_id;

 DBMS_OUTPUT.PUT_LINE('User Name: ' || v_name);

 END print_user;

 FUNCTION get_user_email(p_id NUMBER) RETURN VARCHAR2 IS

 v_email VARCHAR2(100);

 BEGIN

 SELECT email INTO v_email FROM users WHERE id = p_id;

 RETURN v_email;

 END get_user_email;

END pkg_users;

/

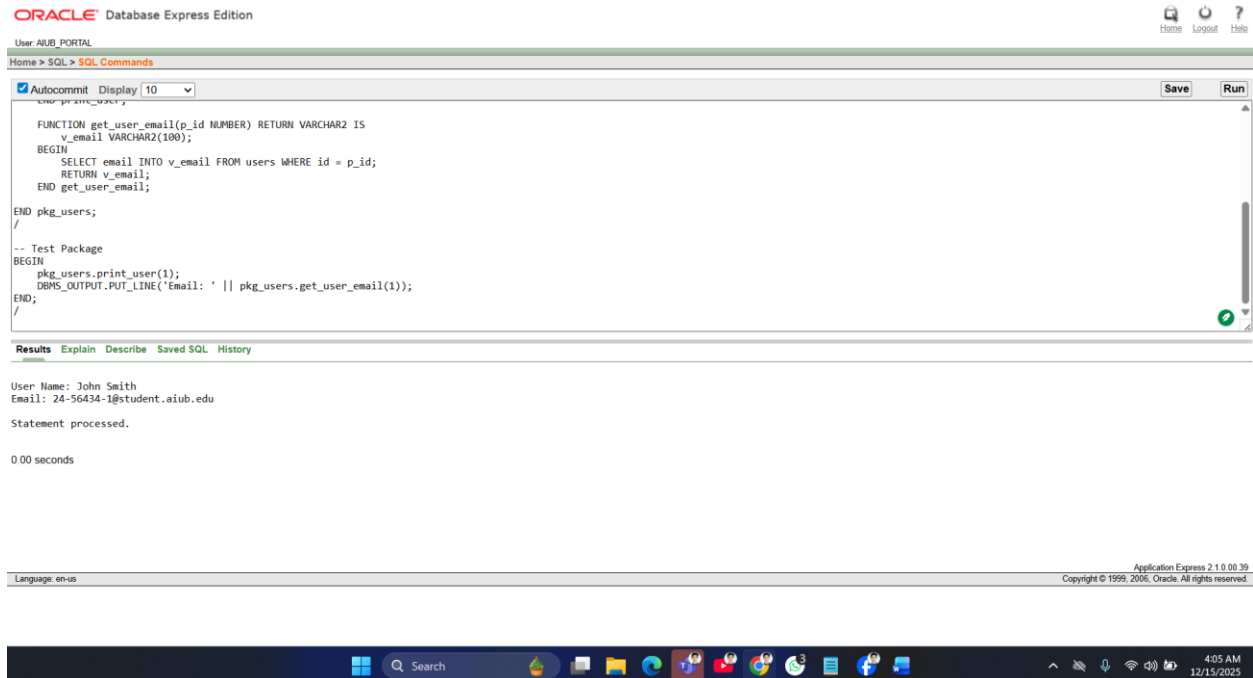
Test Package

BEGIN

 pkg_users.print_user(1);

 DBMS_OUTPUT.PUT_LINE('Email: ' || pkg_users.get_user_email(1));

END;



2. CREATE OR REPLACE PACKAGE pkg_users_info IS

PROCEDURE print_user_department(p_id NUMBER);

FUNCTION get_user_phone(p_id NUMBER) RETURN VARCHAR2;

END pkg_users_info;

/

CREATE OR REPLACE PACKAGE BODY pkg_users_info IS

PROCEDURE print_user_department(p_id NUMBER) IS

v_dept VARCHAR2(50);

BEGIN

SELECT department INTO v_dept FROM users WHERE id = p_id;

DBMS_OUTPUT.PUT_LINE('User Department: ' || v_dept);

END print_user_department;

FUNCTION get_user_phone(p_id NUMBER) RETURN VARCHAR2 IS

v_phone VARCHAR2(20);

BEGIN

SELECT phone_number INTO v_phone FROM users WHERE id = p_id;

RETURN v_phone;


```
END get_user_phone;
```

```
END pkg_users_info;
```

```
/
```

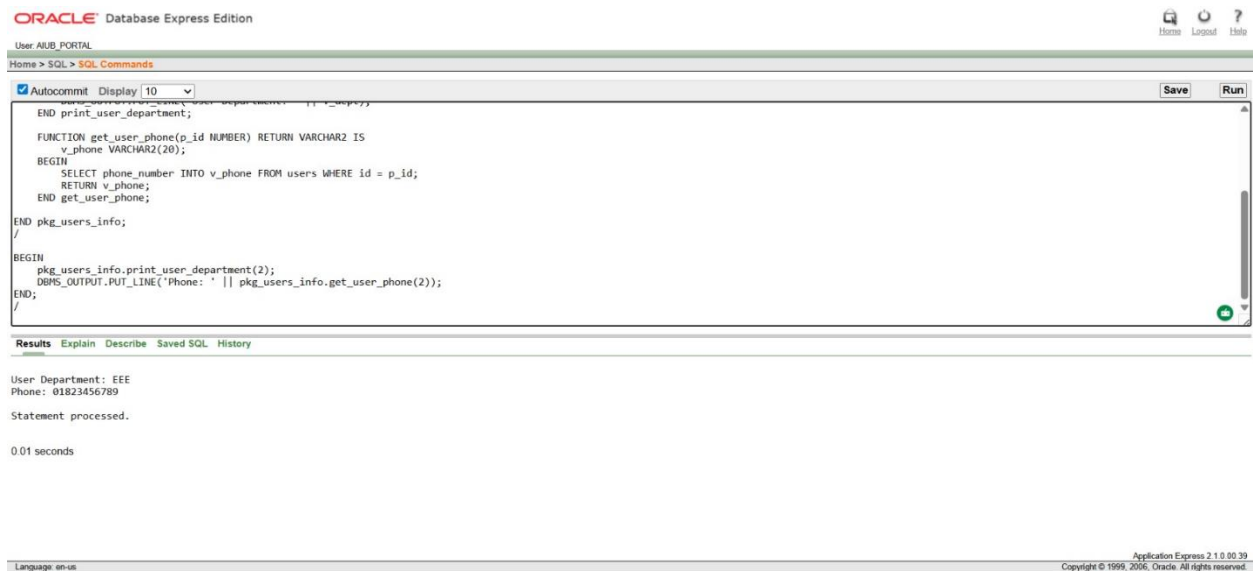
```
BEGIN
```

```
    pkg_users_info.print_user_department(2);
```

```
    DBMS_OUTPUT.PUT_LINE('Phone: ' || pkg_users_info.get_user_phone(2));
```

```
END;
```

```
/
```



Conclusion

The AIUB Sports Portal was created to make sports management at American International University–Bangladesh easier and more organized. It successfully replaces the old manual registration system with a digital platform where students can log in securely, manage their profiles, view tournaments, and register for games online. At the same time, administrators can create tournaments, add games, and manage registrations from one place. This system reduces paperwork, saves time, and helps avoid common mistakes. The portal uses a well-structured database and secure login system to keep data accurate and safe. Overall, the project shows how a simple and well-designed web application can improve the management of university sports activities. In the future, the system can be improved by adding features such as online payment, email or SMS notifications, and match schedules. Live score updates, team management, and result tracking can also be included. Making the portal mobile-friendly or developing a mobile app would make it more convenient for users. With these improvements, the AIUB Sports Portal can become a more powerful and complete sports management system for AIUB.